SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE: CSA3712**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Sl. No	Questions	Bloom's Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3
3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5

4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5

10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

QUESTION 1: Online Shopping – Errors in ECP and BVA

Errors in the original test design

The tester has incorrectly combined valid and invalid inputs in one equivalence class. An equivalence class must contain values expected to produce the same type of system response. If valid prices are accepted but prices outside the permitted range are rejected, they cannot belong to the same class. Mixing them makes it difficult to know whether the test has actually checked the correct behaviour. The second major error is the omission of Boundary Value Analysis (BVA), which is important because defects frequently occur at the edges of an allowed range.

Root cause

The tester has not translated the shopping system's input rules into clear valid and invalid ranges. The test design may have focused on a few typical values and assumed that they represent the entire input domain. However, a price just below the minimum, exactly at the minimum, exactly at the maximum, or just above the maximum may trigger different branches in the validation logic.

Corrected partitions and tests

If the permitted price range is L to U, use three separate classes: (1) price less than L—invalid, (2) price from L through U—valid, and (3) price greater than U—invalid. If the system accepts only numeric prices with a defined decimal precision, malformed text and unsupported precision should also be tested as separate invalid classes.

Test input	Classification	Expected result
L − 1 smallest unit	Invalid: below minimum	Reject with validation message
L	Valid boundary	Accept if other rules pass
L + 1 smallest unit	Valid near boundary	Accept
Typical value between L and U	Valid representative	Accept and calculate correctly
U − 1 smallest unit	Valid near upper boundary	Accept
U	Valid boundary	Accept if other rules pass
U + 1 smallest unit	Invalid: above maximum	Reject

These corrections improve defect detection by ensuring that each class has a clear expected result and that both valid and invalid boundaries are checked. The values L and U must come from the actual requirements; they should not be guessed.

QUESTION 2: Banking Application – Limitations of Random Testing
Errors in the approach

Random testing chooses inputs without systematically targeting requirements or decision boundaries. Therefore, a large number of randomly generated transactions does not guarantee that the minimum transfer amount, maximum daily limit, exact balance boundary, or one unit above a limit will be tested. The QA engineer is incorrectly equating test quantity with test adequacy and claiming high coverage without evidence from requirements or coverage measurements.

Root causes and correction
Critical defects are missed because important boundary values and combinations may be rare in a random sample. Random tests may also be hard to reproduce if the inputs or random seed are not saved. The approach should be corrected by using structured black-box techniques.

Equivalence Class Partitioning (ECP): divide transactions into valid amounts, below-minimum values, above-limit values, insufficient-balance cases, and invalid account states.
Boundary Value Analysis (BVA): test the minimum and maximum transfer values, daily limits, and values immediately below and above them.

Decision Table Testing: combine account status, available balance, transaction amount, and daily limit to test important rule combinations.

Retain random testing only as a supplement for unusual inputs and unexpected combinations; save test data so failures can be reproduced.

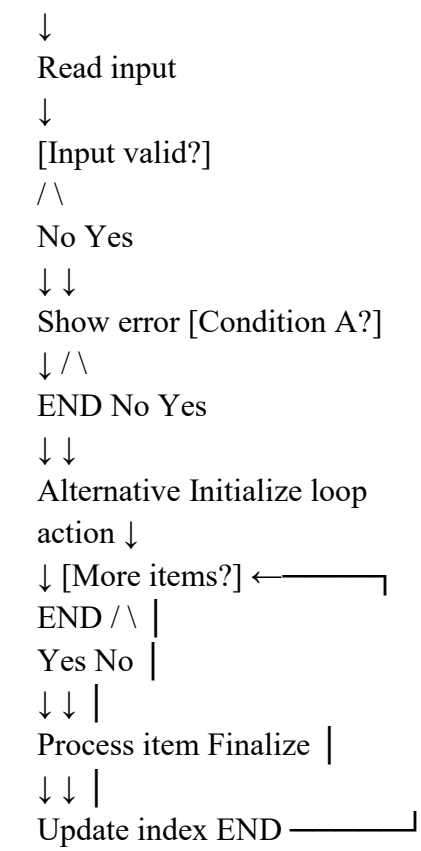
These techniques provide better adequacy because they link tests to the rules of the banking system, ensure that critical boundaries are tested deliberately, and make the results measurable and repeatable.

QUESTION 3: Incomplete Control Flow Graph and Path Testing
Errors in the original CFG
A Control Flow Graph (CFG) represents program execution as nodes and directed edges. The developer has omitted decision nodes and loops, so the graph does not show all True/False outcomes, loop-back edges, or loop-exit paths. Tests designed from this graph will mostly follow the normal route and may fail to execute error handling, alternate actions, or repeated processing. As a result, reported path coverage is unreliable.

Root causes
The source code was not translated statement by statement into graph nodes.
Only the successful execution route was considered; false branches and exception routes were overlooked.

Loop conditions, loop bodies, loop-back edges, and exit edges were not modelled.
The tester did not calculate independent paths or check whether each decision outcome had a test.

Corrected conceptual control flow



This is an illustrative CFG because the assessment does not provide the actual code. The final graph must include every decision, loop, exception, and early return present in the real module.

Independent paths and coverage

Path 1: invalid input → error message → termination.
Path 2: valid input → Condition A is false → alternative action → termination.
Path 3: valid input → Condition A is true → loop condition is false initially → finalize.
Path 4: valid input → Condition A is true → loop executes → item is processed and index updated → loop exits → finalize.

Cyclomatic complexity estimates the number of linearly independent paths. For a single connected graph, $V(G) = E - N + 2$; for a conventional structured graph it can also be calculated as the number of decision nodes + 1. The actual value must be calculated from the completed graph, not from this illustrative drawing.

Improved testing strategy and justification

Construct the CFG from the source, check every edge against the code, identify independent paths, and create at least one test for each feasible basis path. For loops, test zero iterations, one iteration, multiple iterations, normal termination, and relevant limit conditions. Exercise every True and False branch and include invalid input and exception paths. This approach improves coverage because it systematically checks alternate control-flow routes and helps detect incorrect conditions, skipped processing, and loop-termination defects.

QUESTION 4: Statement Coverage but Conditional Errors

Evaluation of the reported coverage

Statement coverage measures whether each executable statement has run at least once. It does not guarantee that every decision outcome has been exercised. For example, if a test makes an if condition true, the statements inside the if body execute, but the else branch may never run. The module can therefore report 100% statement coverage while containing defects in false branches or combinations of conditions.

Coverage gaps and why they matter

Branch coverage gap: a decision's True and False outcomes may not both be executed.
Condition coverage gap: each atomic Boolean condition may not have been evaluated as both True and False.
Path coverage gap: combinations and sequences of decisions may remain untested.
Boundary gap: if a condition checks a limit, values just below, at, and just above that limit may not have been tested.

Corrected example

START → [A is true?]
/ \
Yes No
↓ ↓
Action 1 Action 2
↓ ↓
END END

For this simple graph, one test must make A true and another must make A false. If the decision is compound—for example, A AND B—tests must exercise the atomic conditions appropriately. Condition coverage requires each condition to be True and False at least once; Modified Condition/Decision Coverage (MC/DC) additionally demonstrates that each condition can independently affect the overall decision outcome.

Improved white-box strategy

First, construct and review a complete CFG. Then design tests for each branch outcome and each feasible independent path. Use branch coverage as a minimum improvement over statement coverage, and use condition coverage or MC/DC for complex or high-risk Boolean

expressions. Where loops exist, include zero, one, and multiple iterations and test exit paths. Use coverage tools to identify unexecuted branches, but also check expected outputs because coverage alone does not prove correctness.
The revised strategy improves defect detection by executing the alternative routes that the original tests missed. It is particularly useful for conditional errors, error handling, and decisions where one incorrect Boolean operator can change the system result.

QUESTION 5: Age Validation: Correct ECP and BVA
Errors and cause
The tester defines only two classes—negative numbers and positive numbers. This is incorrect because positive numbers include both valid ages from 1 to 100 and invalid ages greater than 100. The value zero is also not properly represented, even though the stated valid range is 0–100. The absence of BVA means the values at the lower and upper limits and their immediate neighbours are not checked.

Corrected classes and tests
Use three primary equivalence classes: (1) age below 0—invalid, (2) age from 0 through 100 inclusive—valid, and (3) age above 100—invalid. If the field accepts only whole-number ages, fractional and non-numeric inputs should be tested according to the requirements.

Input	Class	Expected result
−1	Invalid below lower limit	Reject
0	Valid lower boundary	Accept
1	Valid near lower boundary	Accept
99	Valid near upper boundary	Accept
100	Valid upper boundary	Accept
101	Invalid above upper limit	Reject
Text/fraction, if unsupported	Invalid format/class	Reject with validation message

This classification separates inputs according to expected behaviour. BVA tests the edges where comparison mistakes commonly occur, improving input validation and reducing off-by-one defects.

QUESTION 6: Condition Coverage – Source Question Is Incomplete
The supplied assessment document is truncated after the text “A developer claims full Condition Coverage for a complex if(A && B”. The remainder of Question 6 is missing. Therefore, the full scenario and required task cannot be answered accurately without inventing information.

Explanation of the visible concept
For a decision if (A && B), the decision is True only when both A and B are True. Condition Coverage requires each atomic condition to take both True and False values at least once across the test suite. One simple set is (A=True, B=True) and (A=False, B=False): A and B each take both values across the two tests.

Test	A	B	A && B
T1	True	True	True
T2	False	False	False

However, this pair alone does not show that A and B independently change the decision outcome while the other condition is held fixed. For stronger Modified Condition/Decision Coverage (MC/DC), a useful set is (A=True, B=True), (A=False, B=True), and (A=True, B=False). Comparing the first test with the second shows A independently changes the decision; comparing the first with the third shows B independently changes it.

QUESTION 7: Discount System – Decision Table Errors

Why the original table is incomplete

The QA team missed combinations of Premium Member (Yes/No) and Coupon Applied (Yes/No). With two binary conditions there are four basic combinations. If one or more are omitted, the system may apply no discount, the wrong discount, or an incorrect combined discount for an untested customer type.

Rule	Premium member?	Coupon applied?	Expected outcome
R1	No	No	Regular price; no member or coupon discount
R2	No	Yes	Apply coupon if valid and eligible
R3	Yes	No	Apply premium-member discount
R4	Yes	Yes	Apply the documented stacking or priority rule

The exact discount percentages and stacking policy are not provided, so Rule R4 must be confirmed from the business requirements. The tester should also check expired or invalid coupons and verify the final amount after rounding. An exhaustive decision table makes missing combinations visible and reduces pricing defects.

QUESTION 8: Loop Testing: Missing n=0 and n=max+1

Errors and root causes

The tester checks only n=1 and n=max. This leaves out n=0, where the loop body should execute zero times, and n=max+1, which may exceed the permitted input limit. A loop can fail before its first iteration because of initialization errors, and it can fail after the last permitted iteration because of an incorrect termination condition. The strategy assumes that first and last allowed iterations are sufficient, but it does not check values outside the normal range.

Corrected approach for a simple loop

n=0: check empty input, initialization, and safe skipping of the loop body.
n=1: check one iteration and correct initial values.
n=2: check repeated execution and index updates.
n=max−1: check behaviour just below the upper limit.
n=max: check the largest permitted iteration count.
n=max+1: check rejection or safe handling of excess input.
Also test early exit, exceptions during an iteration, and termination when the loop condition becomes false.

Nested-loop strategy

For nested loops, begin with the inner loop while keeping the outer loop at its minimum value. Test the inner loop with zero, one, typical, and maximum iterations. Then increase the outer loop and repeat the critical inner-loop cases. Include situations where an inner loop executes zero times for one outer iteration, where the inner loop reaches its maximum, and where an error interrupts processing. Test break/continue logic and ensure indexes are updated correctly.

Path integrity and justification

A CFG should include each loop decision, body, back edge, and exit edge. Branch coverage verifies that the loop condition evaluates both True and False. Boundary tests expose off-by-one defects; zero-iteration tests expose initialization and empty-input problems; over-limit tests verify validation. Exhaustive combinations may be expensive for nested loops, so prioritize independent paths and high-risk boundaries. This strategy improves path integrity by checking entry, repetition, and termination rather than only two selected iteration counts.

QUESTION 9: Safety-Critical Braking System – Error Guessing, BVA and State Transitions

Errors in relying only on Error Guessing

Error Guessing relies on a tester's experience and intuition to anticipate likely failures. It is useful for finding unusual defects, but it is ad hoc and does not guarantee systematic coverage of requirements, input boundaries, or operating states. In a safety-critical braking system, an omitted condition or rare event sequence may have serious consequences. It is therefore unsafe to rely on Error Guessing as the primary method while ignoring structured black-box techniques.

Boundary Value Analysis

Use BVA for every specified threshold, including speed, brake pressure, sensor values, timing limits, and any permitted operating range. For a threshold T, test $T-\varepsilon$, T, and $T+\varepsilon$, where ε is the smallest meaningful input increment. Also test below-minimum and above-maximum sensor values. Expected responses must be based on approved safety requirements; this answer does not assume particular numerical braking thresholds.

State Transition Testing

Normal driving → Brake requested → Braking active → Vehicle slowed/stopped
| |
└──── Sensor/actuator fault ──→ Safe response + warning
Braking active → Fault cleared and checks pass → Controlled recovery

Define states such as normal driving, brake request received, braking active, sensor fault, actuator fault, and safe recovery. Test valid and invalid transitions, including abrupt braking, repeated brake requests, sensor loss during braking, actuator failure, and recovery after the fault clears. Confirm that the system enters the required safe state and reports faults.

Improved safety strategy

Combine BVA with state-transition tests and requirements-based test cases. Add fault injection, hardware-in-the-loop testing, timing checks, independent review, and regression tests. Error Guessing may supplement this structured suite by exploring unusual scenarios, but it should not replace it. The combined approach provides traceability to safety requirements and tests both threshold behaviour and event sequences, improving safety assurance.

QUESTION 10: CFG Missing Else Branches – Correct Graph and Independent Paths

Errors and root causes

The tester has omitted the else branches of nested if statements from the CFG. A decision must show both True and False outcomes, but the incomplete graph represents only the successful route. Consequently, error paths are absent from the derived test set. Statement coverage can still reach 100% if every statement on the successful route executes, while error branches receive no tests and branch coverage remains incomplete.

The root causes are incomplete reading of the source code, failure to represent every decision outcome, and lack of review between the source and the graph. Exceptions, early returns, rollback logic, and nested else branches may have been treated as unimportant even though they change program behaviour.

Corrected conceptual CFG

START
↓
[Input valid?] ─No→ Input error → END
| Yes
↓
[User authorised?] ─No→ Access denied/log → END
| Yes
↓
[Operation successful?] ─No→ Rollback/error → END
| Yes

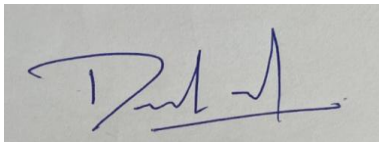
↓
Save result → END
This conceptual graph includes three decisions and separate outcomes. The actual CFG must be adapted to the module's real conditions and include all nested decisions, loops, exception handlers, and early returns. It is not possible to confirm the exact program graph without its source code.

Independent paths and test design
P1: Input invalid → input error → termination.
P2: Input valid → authorization fails → access denied/logged → termination.
P3: Input valid → authorization succeeds → operation fails → rollback/error → termination.
P4: Input valid → authorization succeeds → operation succeeds → save result → termination.
For this simplified graph, three binary decision nodes imply V(G)=4 independent paths. Verify this with V(G)=E−N+2 on the actual graph. Create at least one test for every feasible independent path and check both output and state changes. Branch coverage requires every True/False outcome to execute; path testing checks sequences of decisions.
The corrected CFG exposes missing error routes and supports tests for invalid input, denied authorization, operation failure, rollback, and successful completion. This improves branch adequacy and reduces the risk that logical errors remain hidden behind a 100% statement-coverage figure.

Code of Conduct:

*I **Deepa preya H / 192421047** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand